



Conventional Commits

Um guia completo sobre padrões de commit para desenvolvimento profissional

O Que são Conventional Commits?

- **Padrão internacional** de nomenclatura para mensagens de commit
- **Estrutura clara** que organiza histórico do projeto
- **Automação** de changelog e versionamento
- **Comunicação** clara sobre mudanças no código
- **Profissionalismo** reconhecido na indústria

Índice

- O Que São Conventional Commits
- Função e Benefícios
- Estrutura do Commit
- Tipos de Commits
- Exemplos Práticos
- Boas Práticas
- Guia Passo a Passo
- Erros Comuns
- Conclusão

O Que São Conventional Commits?

Conventional Commits é uma especificação para adicionar significado à mensagens de commit. É uma convenção simples que descreve as mudanças no código de forma estruturada e padronizada.

A especificação foi criada para:

- Estabelecer um **padrão claro** entre desenvolvedores
- Automatizar a **geração de changelog**
- Facilitar a **navegação do histórico** do projeto
- Determinar **incrementos de versão** (Semantic Versioning)
- Comunicar **natureza das mudanças** de forma clara

Benefício Principal:

Um desenvolvedor que nunca trabalhou no seu projeto consegue entender exatamente quais mudanças foram feitas em cada commit, sem precisar ler o código.

Função e Benefícios

Por Que Usar Conventional Commits?

Benefício	Descrição	Impacto
Changelog Automático	Gera automaticamente arquivo CHANGELOG.md	Economiza horas de documentação
Versionamento Automático	Determina se é patch, minor ou major	Usa Semantic Versioning automaticamente
Histórico Legível	Commits organizados por tipo	Fácil navegar e encontrar mudanças
CI/CD Automation	Pipelines reagem a tipos de commit	Deploy automático baseado em commit
Code Review Eficiente	Reviewer sabe o tipo de mudança	Análise mais focada e rápida
Profissionalismo	Padrão reconhecido na indústria	Demonstra qualidade do projeto

Tipos de Benefício

Para o Time

- Comunicação clara
- Menos confusão
- Onboarding mais fácil
- Padronização

Para Automação

- Changelog automático
- Deploy automático
- Versionamento automático
- Triggers para CI/CD

Estrutura do Commit

A estrutura básica de um Conventional Commit segue este padrão:

```
<tipo>(<escopo>): <descrição> <corpo do commit (opcional)> <rodapé (opcional)>
```

Componentes

1. Tipo (Obrigatório)

Define a **categoria da mudança**. Deve ser um dos tipos pré-definidos.

Exemplo:

```
feat , fix , docs , style , refactor , etc.
```

2. Escopo (Opcional)

Define **qual parte do código** foi modificada. Fica entre parênteses após o tipo.

Exemplo:

```
feat(header) , fix(product-card) , docs(readme)
```

3. Descrição (Obrigatório)

Resumo breve da mudança. Deve começar com verbo no imperativo, ser conciso e sem ponto final.

Exemplo:

```
adiciona logo no header (não: "adiciona" ou "adicionou")
```

4. Corpo (Opcional)

Explicação **detalhada** da mudança. Pode ter múltiplas linhas.

Exemplo:

```
feat(auth): adiciona autenticação com JWT
```

Este commit implementa autenticação baseada em JWT:

- Cria middleware de autenticação
- Valida tokens de acesso
- Retorna erro 401 se inválido

5. Rodapé (Opcional)

Referências a **issues**, **breaking changes** ou informações adicionais.

Exemplo:

```
BREAKING CHANGE: mudança de API
```

```
Closes #123
```

```
Refs #456
```

Tipos de Commits

Existem 8 tipos principais de Conventional Commits. Abaixo estão todos com descrição e exemplos:

1. feat - Nova Funcionalidade

Quando você **adiciona uma nova feature/funcionalidade** ao projeto.

Aspecto	Descrição
Significado	Adiciona nova funcionalidade (Feature)
Versionamento	Incrementa versão Minor (x.Y.z)
Changelog	Aparece em "Features"
Frequência	Alta - a maioria dos commits

Exemplos Corretos:

```
feat: adiciona filtro de produtos por categoria
feat(cart): implementa cálculo automático de frete
feat(auth): adiciona autenticação com Google
feat(api): cria endpoint para listar usuários
```

2. fix - Correção de Bug

Quando você **corrige um bug/erro** no código.

Aspecto	Descrição
Significado	Corrige um bug/erro
Versionamento	Incrementa versão Patch (x.y.Z)

Changelog	Aparece em "Bug Fixes"
Frequência	Alta - correções constantes

Exemplos Corretos:

```
fix: corrige alinhamento do header em mobile
fix(product-card): remove duplicação de eventos
fix(api): trata erro de conexão com banco de dados
fix(form): valida email corretamente
```

3. docs - Documentação

Quando você **adiciona** ou **atualiza documentação** (README, comentários, guias, etc).

Aspecto	Descrição
Significado	Muda apenas documentação
Versionamento	Não incrementa versão
Changelog	Geralmente não aparece
Frequência	Média

Exemplos Corretos:

```
docs: adiciona instruções de instalação ao README
docs(api): documenta endpoints de autenticação
docs: corrige typo na documentação
docs(contributing): adiciona guia de contribuição
```

4. style - Formatação de Código

Quando você **muda formatação** (espaços, quebras de linha, etc) sem alterar lógica.

Aspecto	Descrição
Significado	Muda formatação, sem lógica
Versionamento	Não incrementa versão
Changelog	Não aparece
Frequência	Baixa - usar formatadores automáticos

Exemplos Corretos:

```
style: formata código com Prettier
style: remove espaços em branco extras
style(css): padroniza indentação
style: remove ponto-e-vírgula desnecessário
```

5. refactor - Refatoração

Quando você **muda a estrutura do código** sem alterar comportamento.

Aspecto	Descrição
Significado	Muda estrutura, sem função
Versionamento	Não incrementa versão
Changelog	Não aparece
Frequência	Média - melhorias contínuas

Exemplos Corretos:


```
refactor: extrai função reutilizável  
refactor(components): simplifica lógica de renderização  
refactor(api): reorganiza imports  
refactor: renomeia variáveis para clareza
```

6. perf - Performance

Quando você **melhora performance** do código.

Exemplos Corretos:

```
perf: otimiza carregamento de imagens  
perf(api): cache resultados de queries lentas  
perf: remove operações desnecessárias no loop  
perf(bundle): reduz tamanho final do JavaScript
```

7. test - Testes

Quando você **adiciona ou modifica testes**.

Exemplos Corretos:

```
test: adiciona testes para função de validação  
test(auth): testa autenticação com JWT  
test: aumenta cobertura de testes para 80%  
test(api): adiciona testes de integração
```

8. chore - Manutenção

Quando você **faz manutenção** (dependências, configuração, build, etc).

Exemplos Corretos:

```
chore: atualiza dependências
chore(package.json): remove pacote não utilizado
chore: configura husky para pre-commit
chore(build): atualiza webpack para v5
```

Resumo Comparativo

Tipo	Descrição	Versão	Changelog
feat	Nova funcionalidade	Minor ↑	Sim
fix	Correção de bug	Patch ↑	Sim
docs	Documentação	Sem mudança	Não
style	Formatação	Sem mudança	Não
refactor	Refatoração	Sem mudança	Não
perf	Performance	Patch ↑	Sim
test	Testes	Sem mudança	Não
chore	Manutenção	Sem mudança	Não

Exemplos Práticos

Exemplo 1: Commit Simples (Sem escopo)

Correto:

```
feat: adiciona filtro por categoria
```

Incorreto:

```
added new filter feature
```

```
adicionar filtro
```

```
Feature: filtro categorias
```

Exemplo 2: Commit Com Escopo

Correto:

```
feat(product-card): adiciona botão de favoritar
```

Incorreto:

```
feat: adiciona botão de favoritar no product-card
```

```
feat(product_card): adiciona botão favoritar
```

Exemplo 3: Commit Com Corpo

Correto:

```
feat(auth): implementa autenticação com JWT
```

Adiciona autenticação segura usando JWT:

- Cria middleware validateToken
- Valida assinatura do token
- Retorna 401 se token inválido
- Armazena user no request.user

Closes #456

Exemplo 4: Breaking Change

Correto:

feat(api): muda estrutura de resposta da API

BREAKING CHANGE: Response agora retorna objeto 'data'

Antes:

```
{ name: "João", email: "joao@email.com" }
```

Depois:

```
{ data: { name: "João", email: "joao@email.com" } }
```

Exemplo 5: Projeto Real - Syntaxwear

Commits corretos para o projeto Syntaxwear:

```
feat(header): adiciona logo e menu navegação
feat(hero): implementa seção hero com overlay
feat(components): cria componente product-card reutilizável
feat(grid): implementa galeria com CSS Grid responsivo
fix(header): corrige alinhamento em resolução mobile
fix(hero): remove overflow de texto em telas pequenas
docs: adiciona instruções de instalação ao README
refactor(css): organiza variáveis em arquivo separado
style: formata código com Prettier
test: adiciona testes para validação de formulário
```

Boas Práticas

Faça

- **Use imperativo:** "adiciona" (não: "adicionado" ou "adiciona")
- **Seja conciso:** Máximo 50 caracteres na descrição
- **Use escopo:** Quando relevante: feat(header), fix(header)
- **Separe com linha em branco:** Entre descrição, corpo e rodapé
- **Referencie issues:** "Closes #123", "Fixes #456"
- **Use minúsculas:** "feat:" não "Feat:"
- **Sem ponto final:** "adiciona logo" (não: "adiciona logo.")
- **Explique o porquê:** No corpo, explique a razão da mudança

Não Faça

- **Vago:** "atualiza coisas", "faz mudanças"
- **Muito longo:** "adiciona nova funcionalidade incrível que permite usuários fazerem X"
- **Tipo genérico:** "update:", "change:"
- **Misturar responsabilidades:** Um commit = um tipo de mudança
- **Sem contexto:** Código mudou, mas não está claro por quê
- **Passado:** "adicionou", "corrigiu", "implementou"
- **Emojis desnecessários:** Tipo é suficiente
- **Commits quebrados:** Cada commit deve deixar projeto em estado funcional

Dicas de Ouro

Dica 1: Use escopo consistente

Se você tem componente chamado `product-card`, sempre use:

```
feat(product-card): ... (não: product_card ou ProductCard)
```

Dica 2: Um commit = uma mudança

Se você ajustou HTML E CSS, faça dois commits separados:

```
feat(header): adiciona logo
style(header): ajusta espaçamento da logo
```

Dica 3: Commits frequentes

Melhor fazer 10 pequenos commits do que 1 commit gigante com 50 mudanças.

Dica 4: Limpar antes de commitar

Use `git add -p` para selecionar apenas mudanças relevantes.

Guia Passo a Passo - Como Fazer

Passo 1: Faça uma Mudança

```
// Editar arquivo  
vim src/components/header.js  
// Adicionar novo código...
```

Passo 2: Verifique Mudanças

```
git status  
# On branch main  
# Changes not staged for commit:  
#   modified:   src/components/header.js
```

Passo 3: Adicione ao Stage

```
git add src/components/header.js  
# ou adicione tudo:  
git add .
```

Passo 4: Identifique o Tipo

Pergunta-se: "Que tipo de mudança foi feita?"

- Adicionei nova funcionalidade? → `feat`
- Corrigi um bug? → `fix`
- Só atualizei documentação? → `docs`
- Formatei código? → `style`
- Refatorei estrutura? → `refactor`

Passo 5: Identifique o Escopo (opcional)

Pergunta-se: "Qual parte do código mudou?"

- Header? → `feat(header):`
- Product Card? → `feat(product-card):`
- API? → `feat(api):`

Passo 6: Escreva a Descrição

Use imperativo, seja conciso, máximo 50 caracteres:

```
feat(header): adiciona logo no header
```

Passo 7: Faça o Commit

```
git commit -m "feat(header): adiciona logo no header"
```

Passo 8: Se Precisar de Detalhes, Use -e

```
git commit -e  
# Abre editor com template:
```

```
feat(header): adiciona logo no header  
  
Este commit implementa a logo no header:  
- Importa componente Logo  
- Adiciona logo ao lado do menu  
- Logo é clicável e vai para home  
  
Closes #123
```


Passo 9: Envie

```
git push origin main
```

Exemplo Completo

Cenário: Você adicionou um botão de favoritar no product-card

```
$ git add .

$ git status
modified:   src/components/product-card.js
modified:   src/components/product-card.css

$ git commit -m "feat(product-card): adiciona botão de favoritar"

[main 3d88c7f] feat(product-card): adiciona botão de favoritar
 2 files changed, 15 insertions(+), 5 deletions(-)

$ git push origin main
```

x

Erros Comuns e Como Evitar

Erro 1: Usar Tipo Genérico

Errado:

update: mudanças no código

changes: atualizações

fix stuff: várias correções

Correto:

feat(header): adiciona logo

fix(footer): corrige alinhamento

Erro 2: Descrição em Passado

Errado:

feat: adicionou novo filtro

fix: corrigiu bug no header

Correto:

feat: adiciona novo filtro

fix: corrige bug no header

Erro 3: Escopo Errado

Errado:

feat(products-card): ... (deve ser product-card)

feat(ProductCard): ... (case incorreto)

feat(HEADER): ... (maiúscula)

Correto:

```
feat(product-card): ...  
feat(product-card): ...  
feat(header): ...
```

Erro 4: Commits Gigantes

Errado:

```
feat: adiciona header, footer, sidebar e novo sistema de autenticação
```

Correto:

```
feat(header): adiciona componente header  
feat/footer): adiciona componente footer  
feat(sidebar): adiciona componente sidebar  
feat(auth): implementa autenticação JWT
```

Erro 5: Misturar Tipos

Errado:

```
feat: adiciona logo e formata código CSS (mistura feat + style)
```

Correto:

```
feat(header): adiciona logo  
style(header): formata código CSS
```

Erro 6: Sem Contexto

Errado:

```
fix: arruma problema no alinhamento  
feat: muda coisa importante
```

Correto:

`fix(hero):` corrige alinhamento de texto em mobile

`feat(cart):` adiciona frete automático

Conclusão

Resumo Final

- Conventional Commits é um **padrão internacional** para mensagens de commit
- Estrutura: **<tipo>(<escopo>): <descrição>**
- Beneficia: **clareza, automação e profissionalismo**
- Principais tipos: feat, fix, docs, style, refactor, test, chore
- Gera: **Changelog, Versioning e Histórico organizados**

Próximos Passos

-  Adote Conventional Commits hoje mesmo
-  Torne um hábito escrever boas mensagens
- Compartilhe esse documento com seu time
- Configure ferramentas automáticas (commitizen, husky)
- Monitore qualidade do histórico

Ferramentas Úteis

- **Commitizen:** CLI para escolher tipo de commit
- **Husky:** Git hooks para validar commits
- **Commitlint:** Valida mensagens de commit
- **Conventional Changelog:** Gera changelog automático
- **Semantic Release:** Versionamento automático

Dica Final:

Commits bons hoje = histórico legível amanhã = time mais eficiente sempre!

Referência Rápida

```
<tipo>(<escopo>): <descrição> [corpo do commit] [Closes #issue]  
[BREAKING CHANGE: descrição]
```

Tipos Disponíveis

- `feat` - Nova funcionalidade
- `fix` - Correção de bug
- `docs` - Documentação
- `style` - Formatação
- `refactor` - Refatoração
- `perf` - Performance
- `test` - Testes
- `chore` - Manutenção

Exemplos Prontos para Usar

```
feat(componente): adiciona nova funcionalidade  
fix(componente): corrige bug específico  
docs: atualiza README  
style: formata código  
refactor(componente): simplifica lógica  
perf: otimiza performance  
test: adiciona testes  
chore: atualiza dependências
```

Documento: Conventional Commits - Guia Completo

Versão: 1.0

Última atualização: 2024

Baseado em: <https://www.conventionalcommits.org/>

Desenvolvido com: para a comunidade de desenvolvimento